JustifAI

Explainable Credit AI with Legal Justification Engine

"Decisions You Can Trust. Reasons You Can Understand."

Prepared By: Navneeth Arun

Date: May 16, 2026

Version: 1.0 (Draft Publication)

CONFIDENTIALITY NOTE: *This document contains proprietary architecture, machine learning strategies, and system designs intended solely for the development and assessment of the JustifAI project.*

Table of Contents

1. Product Overview

1.1. Executive Summary

The financial technology sector is rapidly adopting artificial intelligence to streamline loan originations and credit scoring. However, the adoption of **opaque, black-box models** has introduced severe regulatory and ethical risks. Consumers are being denied life-altering financial products without actionable feedback. **JustifAI** is engineered to resolve this tension by deploying an **Explainable Credit AI with a Legal Justification Engine**. It bridges the gap between high-speed automated decision-making and human-centric transparency.

1.2. Vision Summary

To construct a fully transparent and accountable credit decisioning system that guarantees **clear, legally grounded explanations** for every automated financial decision executed by the system. We refuse to compromise transparency for automation.

1.3. Mission

To empower modern financial systems to permanently transition from: **Black-Box Decisions:** Where applicants are rejected without actionable feedback or regulatory justification. ***TRANSFORMING TO*** **Explainable, Auditable, and Fair AI Decisions:** Where every metric is quantified, every rule is traceable, and every decision is paired with a human-readable justification.

2. Problem Space & Objectives

2.1. The Problem Statement

Modern credit decision systems are fundamentally broken regarding consumer transparency. They suffer heavily from:

- **Lack of Explainability:** Machine learning models (like deep neural networks or complex ensembles) output binary decisions without detailing *why* the decision was reached.
- **Regulatory Non-Compliance Risks:** Global financial authorities (such as the CFPB in the US and GDPR authorities in Europe) are increasingly demanding algorithmic accountability and the "Right to Explanation".
- **No Audit Traceability:** Inability to reconstruct the exact data state and rule logic at the moment of a historical decision, making regulatory audits a nightmare.
- **Hidden Bias in ML Models:** Undetected prejudice against specific demographics due to skewed historical training data.
- **Poor Customer Trust:** Users are alienated when they are denied credit without understanding **why they were rejected, what factors influenced the decision, or whether the decision was ultimately fair.**

2.2. Objectives

The system is bound by strict, measurable objectives divided into primary operational goals and secondary advanced enhancements.

Primary Objectives:

1. **Provide Accurate Credit Risk Predictions:** Utilize robust Machine Learning algorithms to assess financial viability based on structured input parameters.
2. **Generate Human-Readable Explanations:** Dynamically translate mathematical weights and feature importance scores into plain-language sentences.
3. **Ensure Compliance with Financial Policies:** Hardcode strict regulatory thresholds into a deterministic Rule Engine to override ML suggestions if necessary (e.g., mandatory rejections for active bankruptcies).

4. **Maintain Full Audit Trail:** Persist 100% of transaction payloads, decisions, and logic paths securely in a relational database.

Secondary Objectives:

1. **Detect Bias in Decisions:** Implement statistical monitoring to flag demographic disparities in approval rates.
2. **Support Decision Re-Evaluation:** Allow loan officers to override inputs and simulate alternative outcomes without corrupting the original audit log.
3. **Provide Legal Grounding using RAG:** Implement Retrieval-Augmented Generation to cite specific policy clauses in rejection letters.

2.3. Target Audience Breakdown

The architecture is designed to serve three distinct personas, each with unique access privileges and interface requirements:

1. **Financial Institutions (Loan Officers / Underwriters):**
    - *Usage:* Primary operators utilizing the system for daily automated credit decisioning, portfolio risk assessment, and manual overrides.
2. **Compliance Officers (Internal Auditors / Regulators):**
    - *Usage:* Read-only access to audit historical decisions, review bias detection dashboards, and ensure the institution is protected from algorithmic liability.
3. **End Users (Consumers / Applicants):**
    - *Usage:* The recipients of the final output. They receive an automated, plain-text report explicitly detailing the reasons for their approval or rejection, improving financial literacy and trust.

3. Feature Specifications & User Flow

3.1. Core System Features

The core system is built on a pipeline of sequential modules that process data from ingestion to final justification.

- **Credit Evaluation (The ML Layer)**
  - ***Function**:* Ingests sanitized, structured financial data (income, debt, credit history) and processes it through the trained predictive model.
  - ***Output**:* A raw probability score representing the risk of default (e.g., 0.85 risk).
- **Decision Engine (The Logic Layer)**
  - ***Function**:* Applies deterministic, rule-based logic over the ML score. This ensures that legal minimums (e.g., minimum income rules) cannot be bypassed by ML anomalies.
  - ***Output**:* A categorical decision vector: **APPROVE / REJECT / REVIEW**.
- **Explanation Engine (The Translation Layer)**
  - ***Function**:* Identifies the top features contributing to the ML score (e.g., "High Credit Utilization") and maps them to pre-approved, clear, human-readable reasoning templates.
- **Audit Logging (The Persistence Layer)**
  - ***Function**:* Commits the entire lifecycle of the request to a secure PostgreSQL partition. Stores the input, ML score, triggered rules, and generated explanation for a fully traceable history.
- **Bias Detection (The Fairness Layer)**
  - ***Function**:* Runs asynchronous batch analyses on recent decisions to identify unfair patterns (e.g., flagging if the rejection rate for a specific simulated zip code spikes abnormally compared to the baseline).

- **Appeal Simulation (The Override Layer)**
    - ***Function**:* Allows authorized users to re-run historical decisions with modified inputs (e.g., "What if their income was 10% higher?") without affecting the official audit log.

3.2. Advanced Capabilities (RAG Integration)

- **RAG-based Legal Explanation (The Context Layer)**
    - ***Function**:* Utilizes a localized Large Language Model paired with Retrieval-Augmented Generation (RAG). When a rejection is triggered, the system queries a vector database containing regulatory text to retrieve the exact legal or internal policy clause justifying the rejection, appending it to the customer explanation for maximum transparency.

3.3. End-to-End User Flow

The lifecycle of a single application follows a strict, synchronous sequence:

1. **Data Ingestion:** User submits financial data via the frontend web application.
2. **Risk Evaluation:** The Spring Boot backend routes the sanitized data to the Python ML microservice to evaluate credit risk.
3. **Rule Application:** The Java Spring Boot engine receives the ML score and applies hardcoded financial constraints.
4. **Justification:** The Explanation Engine translates the math and triggered rules into a human-readable format.
5. **Fairness Check:** The Bias module asynchronously analyzes the data point against recent macro trends.
6. **Persistence:** The complete decision matrix is securely stored in the PostgreSQL database.
7. **Delivery:** The final, explainable response is returned to the user interface.

4. Requirements & Metrics

4.1. Functional Requirements (FR)

- **FR1: Input Handling:** The system MUST accept strictly typed, structured JSON payloads representing applicant financial data. All fields must be validated before processing.
- **FR2: Scoring:** The ML service MUST calculate and return a continuous risk score between 0.0 and 1.0, along with an array of feature importance weights.
- **FR3: Decisioning:** The rule engine MUST apply Boolean logic to categorize the score into final statuses. It MUST support minimum thresholds.
- **FR4: Explanation:** The system MUST generate a minimum of two localized text sentences explaining the primary drivers behind the decision, mapped from the feature importance weights.
- **FR5: Logging:** The system MUST asynchronously store 100% of incoming requests, intermediate ML scores, triggered rules, and outgoing responses in an immutable database structure.
- **FR6: Bias Analysis:** The system MUST flag any decision distribution that violates predefined fairness thresholds (e.g., the 80% four-fifths rule).

4.2. Non-Functional Requirements (NFR)

- **Performance:** Core API response time (Data Entry to Final Decision Response) MUST remain **< 500ms** under normal load to ensure a seamless UI experience.
- **Scalability:** The application MUST utilize a **Microservices Architecture** (Spring Boot for logic, Python FastAPI for inference). Both must be containerized via Docker to allow independent horizontal scaling.
- **Security:** The system MUST strictly follow **OWASP Top 10 guidelines**, specifically enforcing input validation, prepared statements for database access, and data sanitization.
- **Reliability:** The architecture MUST feature **fault-tolerant design**, including circuit breakers (e.g., Resilience4j) when communicating with the ML service to prevent cascading failures.

- **Maintainability:** Codebases MUST remain modular, utilizing clear interface boundaries, interface-driven design, and dependency injection.

4.3. Success Metrics & KPIs

To evaluate the operational success of JustifAI, the following Key Performance Indicators (KPIs) will be continuously tracked:

- **Explainability Rate:** Target **100%** of decisions successfully paired with a generated explanation.
- **Explanation Clarity:** Target > 4.0/5.0 on manual/auditor evaluation of explanation readability.
- **Bias Detection Accuracy:** Target zero false negatives in statistical fairness flagging during QA testing.
- **System Latency:** Maintain average API response time below the 500ms SLA.
- **System Uptime:** Target **99.9%** availability for the core decisioning REST endpoints.

5. Project Boundaries & Constraints

5.1. Constraints & Assumptions

To successfully deliver the prototype, several environmental constraints and systemic assumptions have been established:

Strict Constraints:

- **Data Availability: No real banking data will be used.** Due to strict privacy regulations (GDPR, CCPA, GLBA), all models will be trained on structurally accurate but entirely synthetic data.
- **Compute Limits:** The system will operate within the limits of standard local compute resources (Windows / Ubuntu). This requires efficient, lightweight ML models rather than heavy LLMs for the primary scoring loop.
- **Regulatory Scope:** The system will enforce a simulated, simplified set of regulatory rules rather than integrating a full, complex national legal codex.

Core Assumptions:

- **Synthetic Viability:** It is assumed that the synthetic dataset adequately represents real-world financial distributions, multicollinearity, and correlations.
- **Rule Simplification:** It is assumed that complex financial underwritings can be effectively abstracted into programmatic Boolean rules for prototype demonstration.
- **User Baseline:** It is assumed that the end-users possess a baseline understanding of standard financial terms (e.g., "Credit Utilization", "Debt-to-Income").

5.2. Risk Assessment & Future Scope

Identified Risks & Mitigations:

- *Risk:* **Poor ML Model Accuracy** due to synthetic data limitations or lack of feature depth.
    - *Mitigation:* Focus assessment and grading on the *explainability pipeline* rather than raw predictive power.
- *Risk:* **Weak Explanation Quality** appearing robotic, unhelpful, or overly academic.

    - *Mitigation:* Implement dynamic text templating and iterative human review of generated text.
- *Risk:* **Bias Detection Complexity** yielding too many false positives and halting operations.
    - *Mitigation:* Implement adjustable sensitivity thresholds in the bias module.
- *Risk:* **Integration Challenges** between the Java backend and Python ML engine.
    - *Mitigation:* Enforce strict OpenAPI/Swagger contracts between microservices before writing logic.

Future Scope (Post-V1):

- Integration with real-world, anonymized banking datasets via secure data pipelines.
- Full cloud-native deployment using Kubernetes clusters on AWS or Google Cloud.
- Implementation of advanced mathematical explainability libraries (**SHAP/LIME**) to visually chart feature importance on the frontend.
- Integration with live regulatory APIs for real-time compliance updates.

6. System Architecture (HLD)

6.1. High-Level Microservices Overview

JustifAI operates on a robust, **Microservices-based architecture** featuring modular, independent services designed for distinct responsibilities. The ecosystem leverages an Angular frontend, a Java Spring Boot core backend, a Python ML engine, and a PostgreSQL persistence layer.

1. **Presentation Layer (Client Application)**
    - **Technology:** Angular 18+, SCSS, TypeScript.
    - **Responsibility:** Provides the secure dashboard for Loan Officers and the input portal for Applicants. Communicates entirely via RESTful JSON APIs.
2. **API Gateway & Routing (Spring Boot)**
    - **Technology:** Spring Cloud Gateway.
    - **Responsibility:** Acts as the single entry point. Handles JWT token validation, rate limiting, initial payload validation, and request routing. Ensures the internal backend topology remains hidden from the client network.
3. **Core Decision Service (Spring Boot)**
    - **Technology:** Java 21, Spring Boot 3.x, Spring Data JPA.
    - **Responsibility:** The "Brain" of the operation. Orchestrates the workflow. It receives the validated payload from the Gateway, fires an internal HTTP request to the Python ML Service, awaits the score, and then passes the score through the deterministic Rule Engine.
4. **Machine Learning Inference Service (Python)**
    - **Technology:** Python 3.12+, FastAPI, Scikit-Learn, Pandas.
    - **Responsibility:** A lightweight, highly optimized service that loads the pre-trained .pkl model into memory on startup. It receives normalized features and returns a continuous probability score and feature weights in under 50ms.

5. **Persistence Layer (PostgreSQL)**
    - **Technology:** PostgreSQL 18 (Containerized via Docker, mapped to persistent /mnt/data storage).
    - **Responsibility:** Stores all transactional data, audit logs, and user schemas. Optimized for high-volume write operations to support the immutable audit trail.

6.2. Deployment Topology

All backend services are containerized using Docker. The deployment strategy utilizes docker-compose to orchestrate the internal network, ensuring that the Spring Boot services can communicate with the Python ML service and PostgreSQL database without exposing those internal ports to the host machine or the public internet.

7. Technical Design Document (LLD)

7.1. Database Schema Definition

The PostgreSQL database is structured to separate application state from the immutable audit trail. Strict foreign keys and constraints are utilized.

Table: `applications`

This table stores the raw input data for historical reference and model retraining.

- id (UUID, Primary Key, Auto-generated)
- applicant_hash (VARCHAR(255), Not Null) - Anonymized identifier for the user.
- income (NUMERIC(10,2), Not Null)
- credit_utilization (NUMERIC(5,4), Not Null) - Stored as a decimal (e.g., 0.8500).
- missed_payments (INTEGER, Default 0)
- loan_count (INTEGER, Default 0)
- employment_years (NUMERIC(4,1))
- timestamp (TIMESTAMP WITH TIME ZONE, Default CURRENT_TIMESTAMP)

Table: `decision_logs (Immutable)`

This table acts as the system's ledger. Updates/Deletes are strictly forbidden at the application level.

- log_id (UUID, Primary Key)
- application_id (UUID, Foreign Key referencing applications.id)
- raw_ml_score (NUMERIC(5,4), Not Null) - The risk probability between 0.0 and 1.0.
- rules_triggered (JSONB) - Stores an array of rule IDs that fired during evaluation.
- final_decision (VARCHAR(20), Not Null) - Enum: 'APPROVE', 'REJECT', 'REVIEW'.

- explanation_text (TEXT, Not Null) - The generated human-readable justification.
- created_at (TIMESTAMP WITH TIME ZONE, Default CURRENT_TIMESTAMP)

7.2. Component Interactions & API Contracts

The interaction between the Spring Boot *Core Decision Service* and the Python *ML Inference Service* is standardized using JSON over HTTP.

Endpoint: POST http://ml-inference-service:8000/api/v1/predict

Request Payload (Spring -> Python):


```
{
        "application_id": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
        "features": {
            "income_normalized": 0.45,
            "util_ratio": 0.82,
            "missed_payments": 1,
            "emp_years": 4.5
            }
}
```


Response Payload (Python -> Spring):


```
{
 "application_id": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
 "risk_score": 0.7850,
 "risk_category": "HIGH",
 "top_influencers": [
  {
   "feature": "util_ratio",
   "weight": 0.41,
```

```
    "impact_direction": "NEGATIVE"
   },
   {
    "feature": "missed_payments",
    "weight": 0.35,
    "impact_direction": "NEGATIVE"
   }
  ],
  "model_version": "v1.0.0-logreg"
}
```


Logic Flow: The Spring Boot service receives this JSON, verifies the model_version, and maps the top_influencers directly into the Explanation Engine's localization templates to generate the final human-readable string.

8. Security & Compliance Strategy

8.1. Threat Model & Security Controls

This segment defines the security architecture, data protection strategies, compliance considerations, and risk mitigation approaches. The financial nature of JustifAI demands a paranoid, defense-in-depth approach to system design.

Potential Threats:

1. **Injection Attacks (SQL/Command):** Malicious payloads attempting to alter database queries or execute shell commands on the server.
2. **Data Exposure:** Sensitive data leakage through improper error logging, unhandled exceptions returning stack traces, or unencrypted storage.
3. **API Abuse:** Automated botnets spamming the evaluation endpoint to reverse-engineer the ML model's weights via mass trial-and-error.
4. **Model Exploitation:** Adversarial inputs (e.g., sending negative income values or extreme outliers) designed to trick the logistic regression boundaries into forcing an approval.

Security Controls & Mitigations:

- **Input Validation:** Validate all incoming JSON payloads against strict schemas using Spring Boot @Valid and Jakarta Validation annotations. Reject malformed inputs immediately at the Gateway level before they hit application logic.
- **Authentication (Phase 2):** Implement JWT-based (JSON Web Token) authentication. Token validation will occur exclusively at the API Gateway.
- **Authorization:** Enforce Role-Based Access Control (RBAC). A standard user cannot access the /api/audit endpoints reserved for Compliance Officers.
- **Data Encryption:**
    - *In Transit:* Force HTTPS (TLS 1.2+) for all external traffic.
    - *At Rest:* Future enhancement to encrypt sensitive fields at the block-storage level.

- **Secure Logging: CRITICAL MANDATE:** Do NOT log raw income, account numbers, or personal identifiers to standard out (stdout) or log files. Log only hashed metadata and transaction IDs.
- **Rate Limiting:** Implement token-bucket rate limiting using Spring Cloud Gateway Redis Rate Limiter to prevent API abuse.
- **Error Handling:** Catch all exceptions globally via @ControllerAdvice. Return standardized, generic JSON error messages to the client.

8.2. OWASP Implementations & Auditing

Compliance Considerations:

- **Explainability Compliance:** Every decision record stored in the database **must** include the generated explanation text. A null explanation constitutes a system failure and must trigger a rollback.
- **Audit Compliance:** Maintain a full, append-only decision history. UPDATE and DELETE queries are disabled for the decision_logs table user.
- **Fairness & Bias:** The Bias module runs statistical parity checks on a scheduled cron job to ensure fairness across demographic proxy groups.

OWASP-Based Security Practices (Top 10):

- **A1: Injection:** Exclusively use parameterized queries via Spring Data JPA/Hibernate. String concatenation for SQL building is strictly forbidden.
- **A2: Broken Authentication:** Utilize secure, short-lived JWT tokens with secure HTTP-only cookies.
- **A3: Sensitive Data Exposure:** Avoid logging sensitive info; use environment variables (.env) injected at runtime for all database credentials and secret keys.
- **A4: Broken Access Control:** Enforce strict RBAC annotations (@PreAuthorize("hasRole('COMPLIANCE_OFFICER')")) on administrative Controllers.
- **A5: Security Misconfiguration:** Disable default Spring/Tomcat error pages to prevent server version leakage; close unused Docker ports.

- **A6: Vulnerable Components:** Enforce automated dependency scanning to keep NPM and Maven packages updated.

9. Machine Learning & Data Pipeline

9.1. Dataset Design & Feature Engineering

This document defines the mathematical and data-driven core of JustifAI. The primary objective of the Machine Learning component is to predict a **credit risk score** based on user financial data.

Data Source:

Due to privacy and regulatory constraints, real financial data is unavailable. The system will rely on a highly calibrated **synthetic dataset** generated using statistical distributions (e.g., using Python's Faker and numpy.random libraries) to simulate realistic financial behavior.

Input Variables (Features):

1. **income**: Monthly verified income.
2. **credit_utilization**: Percentage of available credit currently utilized by the user.
3. **missed_payments**: Total count of late or missed payments over a 24-month period.
4. **loan_count**: Total number of currently active, outstanding loans.
5. **employment_years**: Continuous years of employment at current job.

Data Preprocessing & Engineering:

Raw data cannot be fed directly into the model. The Python pipeline handles:

- **Imputation:** Handling missing values by applying the median value of the respective column.
- **Normalization:** Applying Min-Max Scaling so large income numbers (e.g., 50000) do not overshadow small utilization percentages (e.g., 0.85) in the gradient descent calculations.
- **Derived Features (Crucial for Accuracy):**
    - *Debt-to-income ratio (DTI):* Calculated feature linking income to loan counts.

- *Payment reliability score:* A mathematical decay formula reducing the weight of older missed payments compared to recent ones.

9.2. Model Training, Evaluation, & Integration

Initial Model Architecture:

The system will deploy a **Logistic Regression** model as the primary scoring engine.

- *Why Logistic Regression?* Explainability is the core mission. Deep Learning models are inherently "black boxes." Logistic Regression provides explicit, readable coefficients (weights) for every single feature. This makes it mathematically trivial to determine exactly *why* a specific score was generated—a strict requirement for feeding the Explanation Engine.

Training Process:

1. **Split:** Divide the synthetic dataset into 80% Training Data and 20% Holdout Test Data to prevent overfitting.
2. **Train:** Fit the Logistic Regression algorithm to the training split using scikit-learn.
3. **Evaluate:** Score the model against the holdout set using cross-validation.
4. **Export:** Serialize the trained model and the scaler objects into a robust .pkl (Pickle) file.

Evaluation Metrics:

The model's success is quantified via a Confusion Matrix, optimizing for:

- **Accuracy:** Overall correctness across all predictions.
- **Precision:** Minimizing false positives (e.g., incorrectly approving high-risk individuals).
- **Recall:** Minimizing false negatives (e.g., incorrectly rejecting low-risk individuals).
- **F1 Score:** The harmonic mean of Precision and Recall, providing a balanced metric for uneven datasets.

Integration Strategy:

The .pkl model will NOT be loaded directly into the Java application.

- **Adopted Architecture:** Expose the Python model via a dedicated FastAPI REST endpoint.
- **Reasoning:** This thoroughly decouples the ML environment from the backend application. Python is the industry standard for data science; running a separate microservice allows data scientists to update, retrain, version, and deploy new models rapidly without requiring the Java engineering team to recompile and redeploy the Spring Boot backend.